Computer Science Extended Essay

Research Topic:

Exploring approaches to train multiple variable linear regression models

Research Question:

To what extent can K-Fold Cross Validation be used to train a multiple variable linear regression model to accurately predict Singapore's Private Real Estate Price Index value?

Word Count: 4000

Table of Contents

1. Introduction

1.1 Rationale

The modern world can be characterized by one word: data. From mobile devices to IoT (Internet of Things) gadgets and many more, the digital world has seen a massive increase in the amount of data being generated. This data boom has been given a name- *Big Data*, or data that cannot be fully processed using traditional methods.

One sector of society which has always been data centric and has only been catalysed further by *Big* Data is the financial sector. From stock price movements to currency value changes or anything related to our economies, the financial sector pours out continuous streams of data every second of the day, every day of the year. Even the most distinguished professionals cannot make sense of all the data being generated, let alone the millions of employees in the industry. This raises a pertinent question- if data is being generated at a rate incomprehensible to the masses, *why bother expending precious human resources on such data collection?*

This is where the interdisciplinary field of computer science steps in, providing a medium to process all this information and allowing us to mobilize the voluminous quantities of data generated in the digital world. Owing to *Big Data*, one particular subfield of computer science significantly gained traction in the 21st century- *machine learning*.

Putting two and two together, I was intrigued to explore where machine learning could be applied in my local economy, which led me consider its potential application in predicting Singapore's private real estate price index. The ability to predict its value using commonly accessible financial data- equity market information, for example- would aid potential investors in making informed decisions concerning when to buy or sell private real estate.

1.2 Aim

This essay will analyse the efficacy of K-Fold Cross Validation, a model training technique, by applying it to develop several multiple variable linear regression models to predict Singapore's private real estate index value using equity market data. The accuracy of predictions made by the models trained using K-Fold Cross Validation will thus persuade or dissuade its potential real world use case in the fintech industry.

2. Background Information

2.1 Machine Learning

2.1.1 Overview

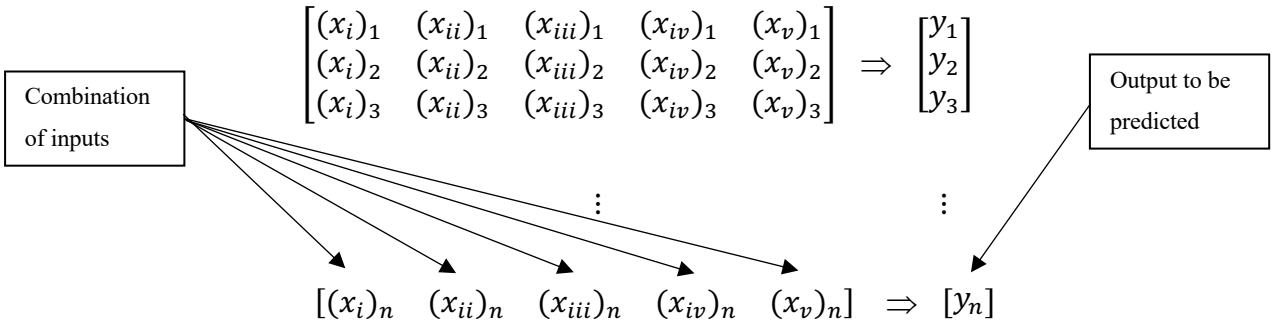
The large subfield of Artificial Intelligence (AI) in the domain of computer science, machine learning is a *form of AI that makes predictions from data*[1]. Machine Learning's primary goal concerns fitting models that categorize large amounts of data in forms understandable by programmers and common people, allowing for predictions (based on the type of model) after *learning* to do so from input data. Unlike traditional algorithms, machine learning algorithms are not explicitly programmed what to output- they make use of statistical analysis tools to output dynamically-changing values. The increased amounts of data available to train prospective ML models have increased their accuracies, to the extent that several new real-world applications have started to emerge.

Due to the approximately linear relationship between the input variables (equity market indices) and the value to be predicted (private real estate index), this essay will explore and compare K-Fold Cross Validation with Train-Test Splitting by applying them in a continuous value prediction (regression) model, known as *multiple variable linear regression*.

[1] *What is Machine Learning? (n.d.). Hewlett Packard Enterprise. Retrieved January 12, 2022, from https://www.hpe.com/sg/en/what-is/machine-learning.html*

2.1.2 Multiple Variable Linear Regression

Multiple variable linear regression is a supervised machine learning algorithm, training a model based on input-output pairs with multiple numeric inputs corresponding to a single output. The algorithm *learns* how to predict, for a combination of inputs, their corresponding output, by training on multiple such rows of data.



The example above details a situation with five input variables to each output. For multivariate linear regression, however, there can be any integer number of inputs greater than 1.

Using an optimization algorithm, each input category is assigned a coefficient, indicating it's *weightage* on the output, i.e. how much of an influence that particular input has on the generated output.

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_i X_i \textit{ (where } \{X_1, X_2, \ldots X_i \rightarrow Y\}\textit{)}$$

The formula above is the defining equation of multiple variable linear regression, with inputs X_1 through X_i mapping to an output *Y*. β_0, β_1, β_2, etc. represent the coefficients (calculated by the optimization algorithm) assigned to the inputs X_1, X_2, X_3, … for all *i* inputs. The coefficient values generated by the model change based on the number of input-output groups fed to the algorithm- thus, the greater the amount of input data, the

greater accuracy of coefficient generated. The algorithm's nomenclature stems from the linear nature of the input-output relation.

Visually representing this, when the datapoints are plotted on a graph, the linear regression model attempts to fit a trendline of best fit to cover as many datapoints as possible, as shown on *Figure 1.1*.

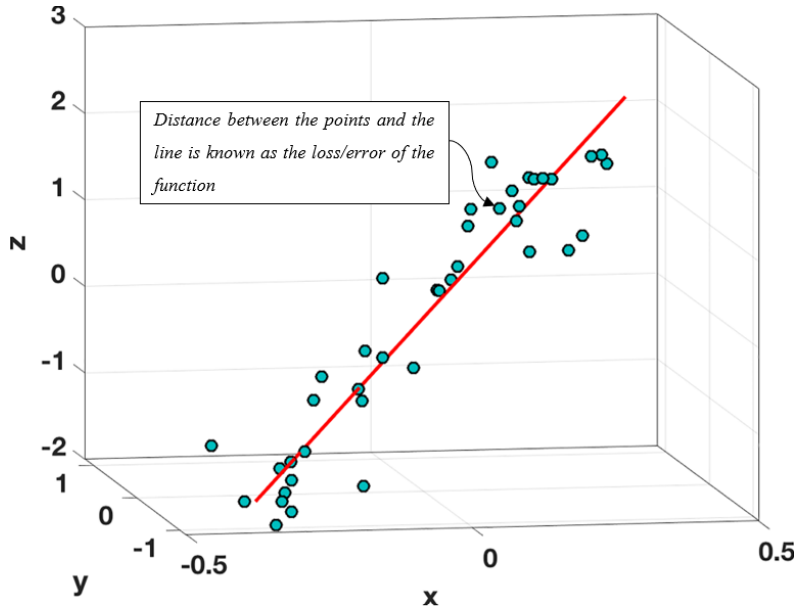


Figure 1.1- Multiple Variable Linear Regression (graphically represented)

2.1.3 Training the model: K-Fold Cross Validation

The simplest method to train a multivariate linear regression model is using train-test splitting, where the input data is split into a training set for model development, and a testing set to observe prediction accuracy. While the size of the training and testing groups can be altered, there is always a large disparity in accuracy and the risk of model coefficients becoming overfit to data. To avoid these problems, a different training technique can be applied- *K-Fold Cross Validation.*

K-Fold Cross validation (CV) is a statistical method used to develop machine learning models. Easy to implement with a lower *bias* than other methods, K-Fold overcomes the issue of overfitting and gives an accurate idea of how the program will perform in general.

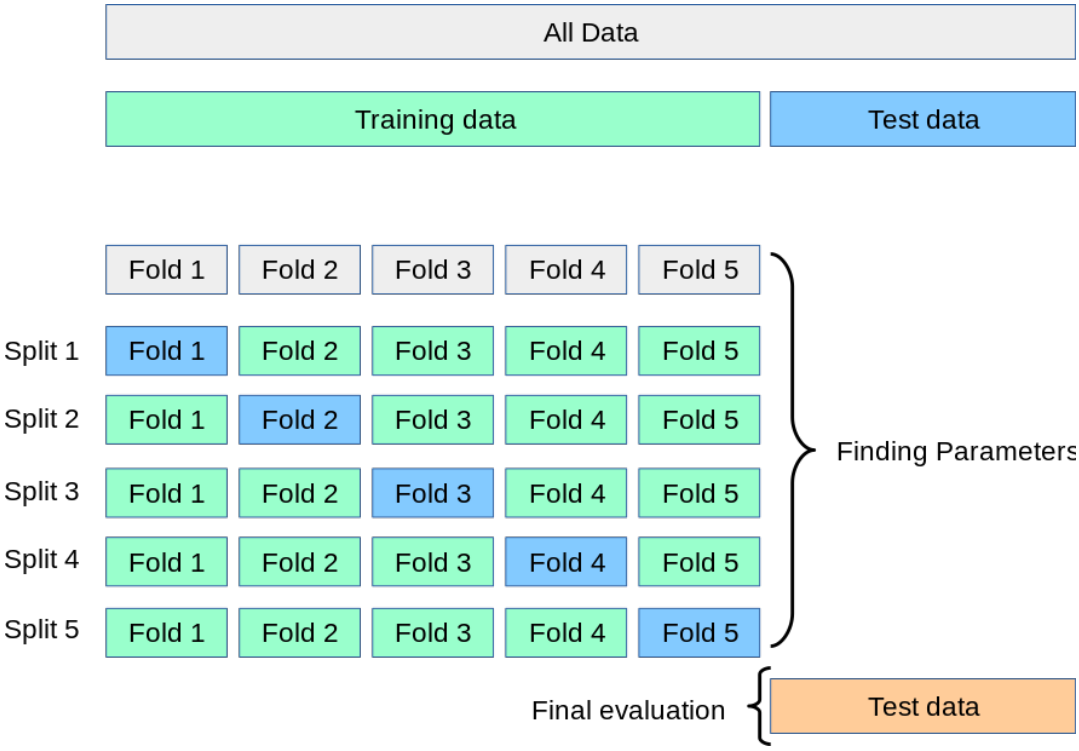


Figure 1.2- K-Fold Cross Validation[2]

Figure 1.2 gives a general description of how K-Fold works. After an initial train-test split, the training data is further split into *k* groups, *k* number of times. In each split, one of the folds' is left for testing, while the other *k-1* folds are used to train and obtain coefficients. Models employing K-fold can be configured to record and store the optimum split, such that these optimum coefficients from that split are retained to be used on the testing set, thus yielding the most accurate possible model.

K-Fold Cross Validation itself can be configured for different circumstances, or simply greater accuracy:

[2] *3.1. Cross-validation: evaluating estimator performance. (n.d.). Scikit-Learn. Retrieved January 11, 2022, from https://scikit-learn.org/stable/modules/cross_validation.html*

2.1.3.1 Repeated K-Fold Cross Validation

Repeated K-Fold is a technique where the basic K-Fold CV is repeated an additional number of times as defined by the programmer. The purpose of this is to obtain a potentially better split than the ones produced in the first folds.

2.1.3.2 Stratified K-Fold Cross Validation

Stratified K-Fold is an extension to K-Fold CV in which the folds are selected such that the mean response value is approximately equal in all folds. Stratified K-Fold ensures that the percentages of samples of each class present in the training data is uniform across all folds.

Nevertheless, simply applying K-Fold Cross Validation isn't enough to train a model- while it helps split the data for training and testing, the training data still needs to be applied in an *optimization algorithm* to calculate the model parameters (coefficients/weights), and therefore the regression line. There exist several optimization algorithms which can do so, such as gradient descent, normal equation optimization, etc., but this exploration has been conducted using python's *sklearn* machine learning library, which makes use of *Ordinary Least Squared (OLS)* optimization.

2.1.4 Ordinary Least Squared (OLS) Optimization

As previously mentioned, the genesis of simple linear regression is finding parameters which minimizes the errors. *Ordinary Least Squared error* minimization achieves this by minimizing the squared errors, removing the possibility of the positive and negative errors cancelling each other out.

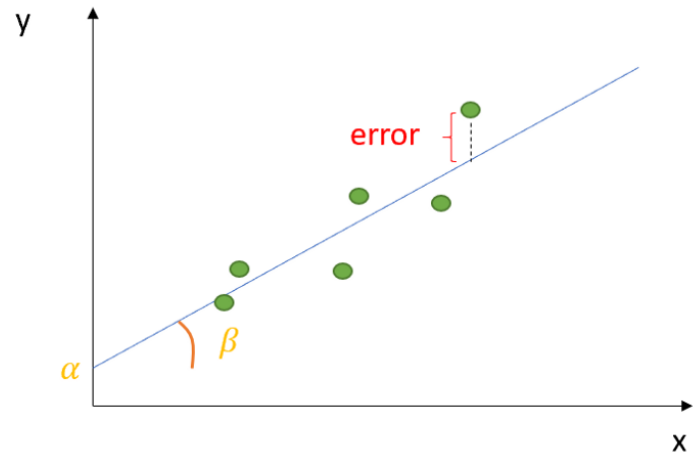


Figure 1.3- OLS for a 2-coefficient situation

The graph above is a visual depiction of the OLS model, simplified for a situation in which there are two optimum coefficients to be found (∝, β, where $y_i = \propto + \beta x_i$). Considering the defining equation of multivariate linear regression to be optimized, the OLS squared error sum becomes

$$\sum_{i=1}^{n}(y_i - \propto - \beta \times x_i)^2 = S(\propto, \beta)^3$$

The equation above signifies that in an optimum regression situation, each y_i value is perfectly mapped by $\propto + \beta x_i$, such that $y_i - (\propto + \beta x_i) = 0$. Applying partial derivatives to this optimization problem, the optimum coefficient values can be expressed in terms of the individual x&y values and their means ($\bar{x}$, $\bar{y}$):

Coefficient Values are dependent on each other, as can be seen from ∝ equation

$$\beta = \frac{\sum_{i=1}^{n}(y_i - \bar{y}) \times (x_i - \bar{x})}{\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

$$\propto = \bar{y} - \beta \times \bar{x}$$

[3] *Alto, V. (2021, December 11). Understanding the OLS method for Simple Linear Regression. Medium. Retrieved January 11, 2022, from https://towardsdatascience.com/understanding-the-ols-method-for-simple-linear-regression-e0a4e8f692cc*

The optimization algorithm is sound in theory, but when applied experimentally, it needs to be checked for accuracy:

2.1.5 Accuracy Measurement

In addition to the variance score[4] calculated by the *sklearn* library, two other accuracy checks have been employed to check for model accuracy, detailed below.

2.1.5.1 Mean Squared Error (MSE)

MSE is calculated by dividing the sum of the square of the distance between the model boundary/regression line and each data point (removes negative/positive bias).

$$\text{MSE} = \frac{1}{n} \times \sum_{i=1}^{n} (y_i - \bar{y}_i)^2$$

2.1.5.2 R-Squared Error

In statistical terms, R-Squared (R^2) error is a measure known as the "coefficient of determination[5]" and is another indication of the correctness of fit of a set of predictions. Like the variance score, the value lies between 0 and 1. The R^2 value calculation is partially linked to MSE:

$$\textit{Total Sum of Squared (TSS) (= } n \times \textbf{\textit{MSE}}\textit{)} = \sum_{i=1}^{n} (y_i - \bar{y}_i)^2$$

$$\textit{Explained Sum of Squares[6] (ESS)} = \sum_{i=1}^{n} (\hat{y}_i - \bar{y}_i)^2$$

$$\Rightarrow R^2 = 1 - \frac{ESS}{TSS}$$

[4] *Variance score lies between 0 and 1, with 1 indicating a perfect prediction model and 0 being a perfectly inaccurate model.*
[5] *Alto, V. (2021a, December 11). Simple Linear Regression with Python - DataDrivenInvestor. Medium. Retrieved January 11, 2022, from https://medium.datadriveninvestor.com/simple-linear-regression-with-python-1b028386e5cd*
[6] *ESS is the sum of the squares of the differences between the deviations (ŷ) and the mean (ȳ, regression line)*

Accuracy is not the sole factor differentiating regression models, they need to be accurate as well as *efficient*- this can be found by measuring the runtime of a given model, calculated using *time complexity*.

2.1.6 Time Complexity

Time complexity refers to the amount of time taken by an algorithm to run. Represented using the *O(n)* notation, it is affected by the length of the input set on which the algorithm operates. The relationship between the input length and time complexity can be linear, quadratic, cubic, exponential, logarithmic, etc.

2.2 Case Study: Equity Markets Indices and Singapore's Private Real Estate Index[7]

2.2.1 Singapore's Private Real Estate Price Index

The essay's primary goal is prediction of the private real estate price index in Singapore, which allows prospective buyers to estimate the average price of their potential real estate purchase.

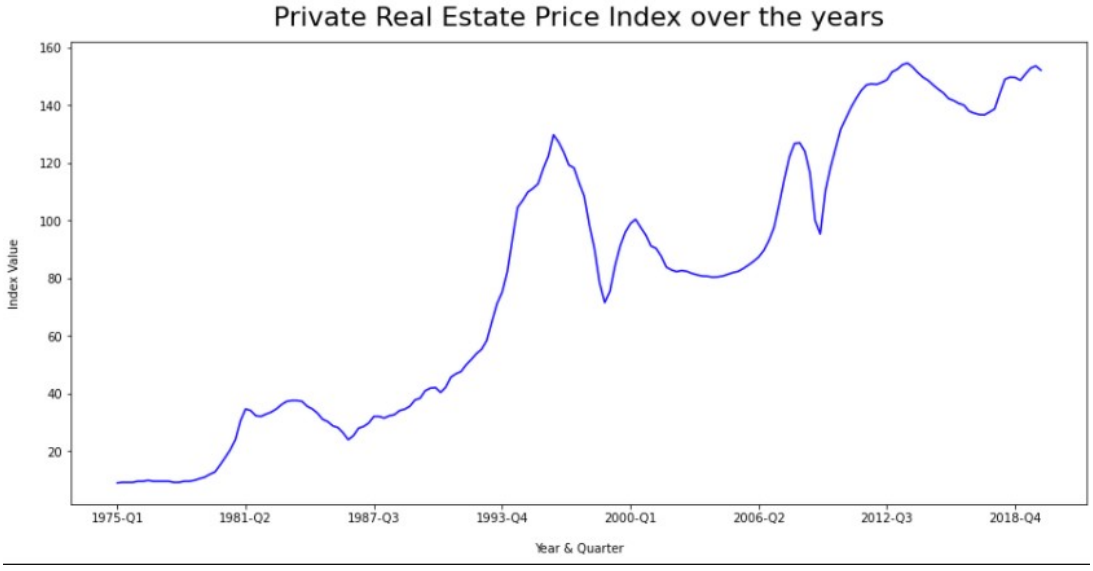


Figure 2.1- Singapore's Private Real Estate Price Index (1975-2019)

[7] *Figures 2.1-2.6 were plotted using Python's matplotlib library*

2.2.2 Equity Market Indices

Market indices are portfolios of investment holdings that represent a segment of the financial market. For the purposes of this essay, five major equity market indices will be considered, with the quarterly values from 1975 to 2019 being used as input data to generate the multivariate linear regression model predicting Singapore's private real estate price index.[8]

2.2.2.1 Dow Jones Industrial Average Index

Dow Jones Industrial Average is a stock market index that tracks 30 of the largest public limited companies that trade on the New York Stock exchange. The index tracks "blue-chip companies", or *nationally recognized, well-established, and financially sound companies*[9]

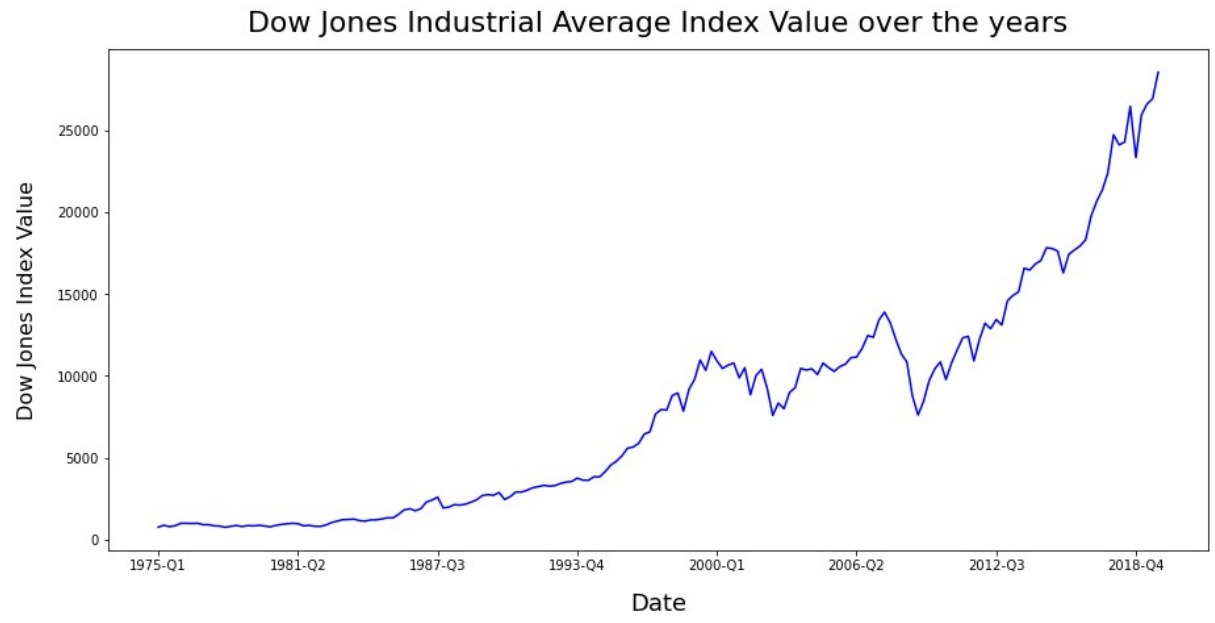


Figure 2.2- Dow Jones Industrial Quarterly Index Value (1975-2019)

[8] *Figures 2.1 – 2.6 were plotted using Python's matplotlib library*

[9] Chen, J. C. (2020, December 27). *What is a Blue Chip?* Investopedia. Retrieved December 13, 2021, from https://www.investopedia.com/terms/b/bluechip.asp

2.2.2.2 S&P 500 Index

The S&P 500, short for the Standard & Poor's 500 Index, is a weighted index tracking the economic activity of the 500 largest public limited companies in the United States.

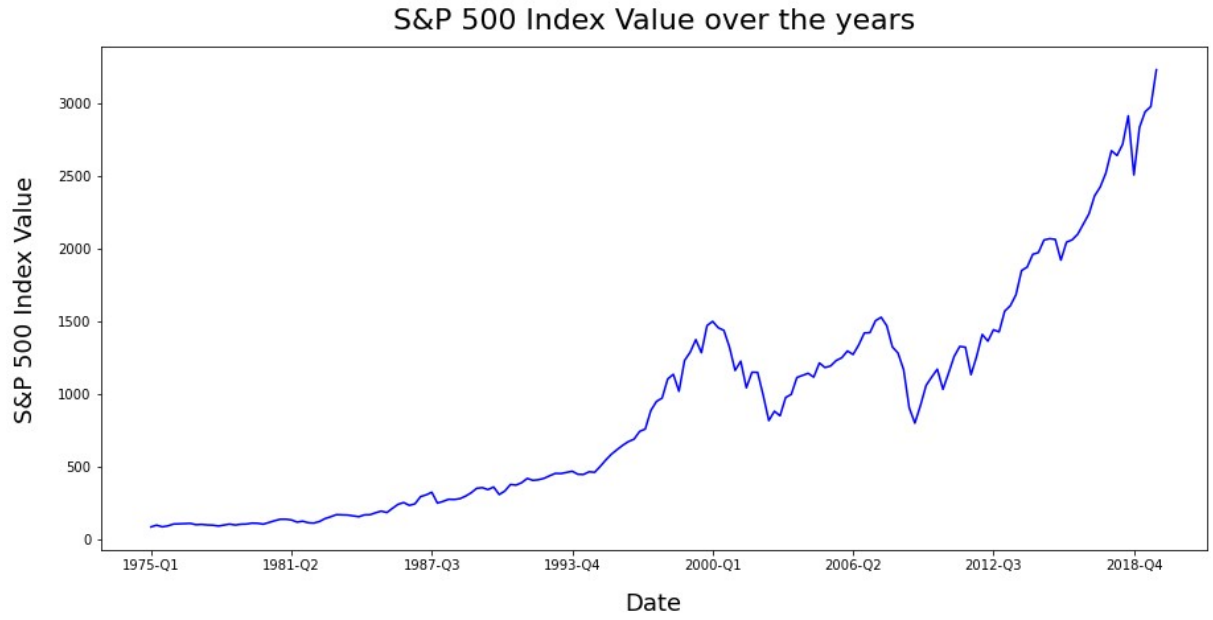


Figure 2.3- S&P 500 Quarterly Index Value (1975-2019)

2.2.2.3 Nasdaq Composite Index

The Nasdaq Composite Index is a weighted index of around 2500 equities listed on the Nasdaq stock exchange, *a global electronic marketplace for buying and selling securities*[10], such as American depository receipts, common public company stocks, Real Estate Investment Trusts (REITs), etc.

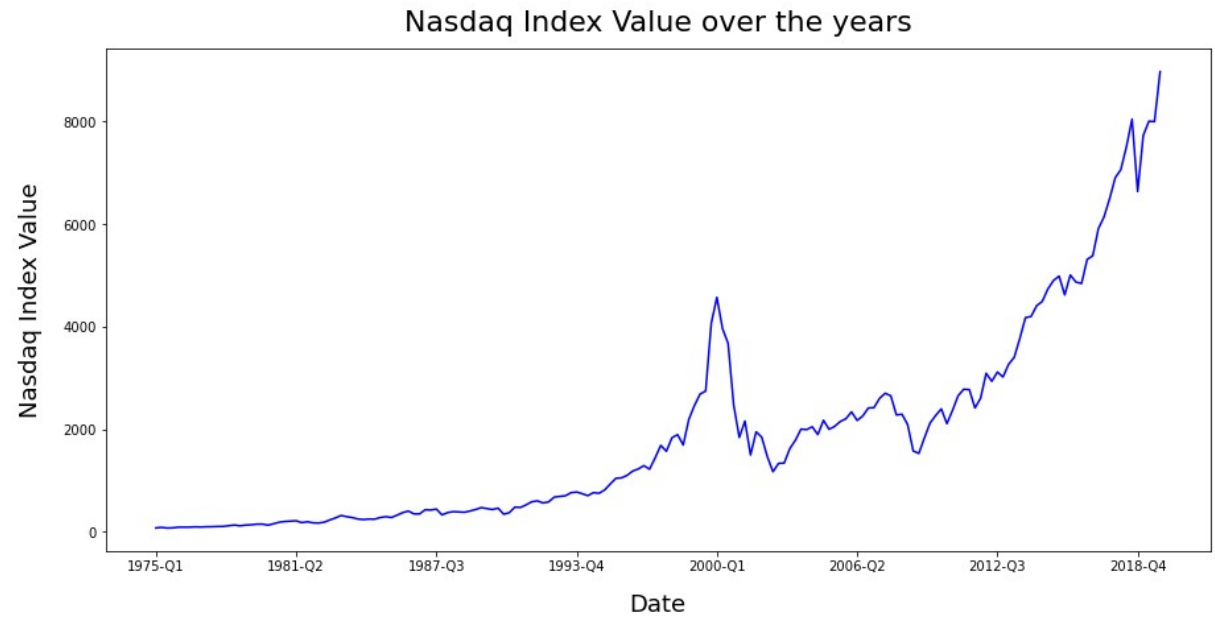


Figure 2.4- Nasdaq Composite Quarterly Index Value (1975-2019)

[10] *Hayes, A. H. (2022, January 8). What Is Nasdaq? Investopedia. Retrieved December 13, 2021, from https://www.investopedia.com/terms/n/nasdaq.asp*

2.2.2.4 Nikkei 225 Index

The Nikkei 225, also known as *Nikkei* or *Nikkei Index*, is a stock market index covering the performance of the 225 largest public limited companies on the Tokyo Stock Exchange (TSE).[11]

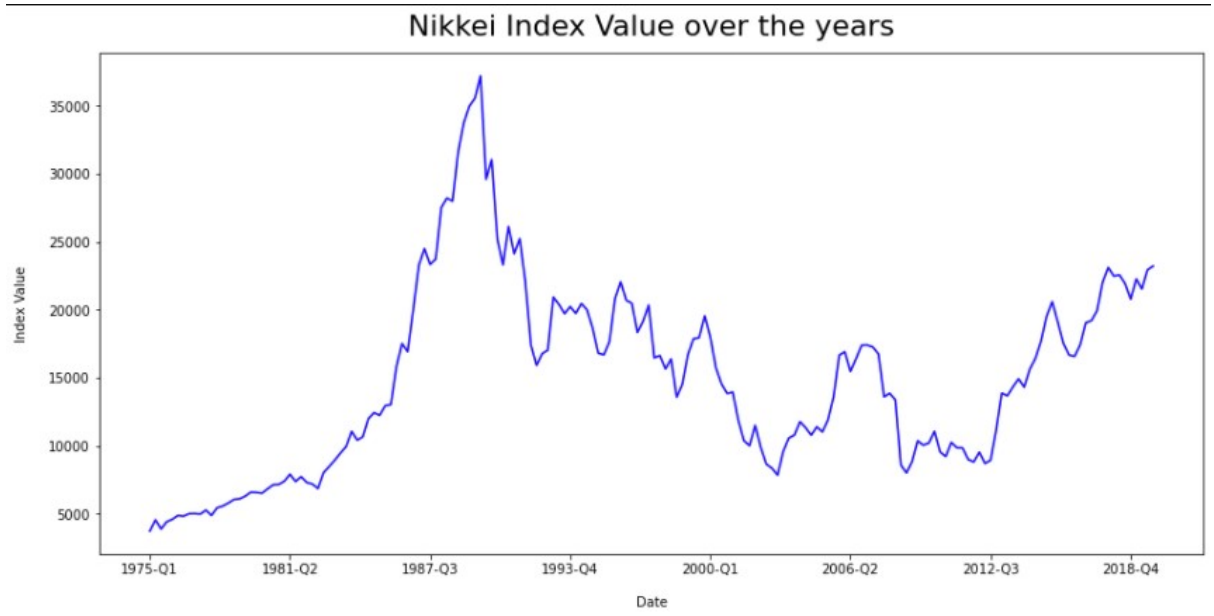


Figure 2.5- Nikkei 225 Quarterly Index Value (1975-2019)

[11] *Nikkei 225 operates on the Japanese Yen- values have been adjusted to USD to adjust for potential effects on value prediction.*

2.2.2.5 NYSE Composite Index

The NYSE Composite Index is an index measuring the performance of all (common) stocks listed on the New York Stock Exchange. The sheer number of stocks and therefore businesses the NYSE Composite Index accounts for makes it a better indicator of market performance compared to narrower indices.

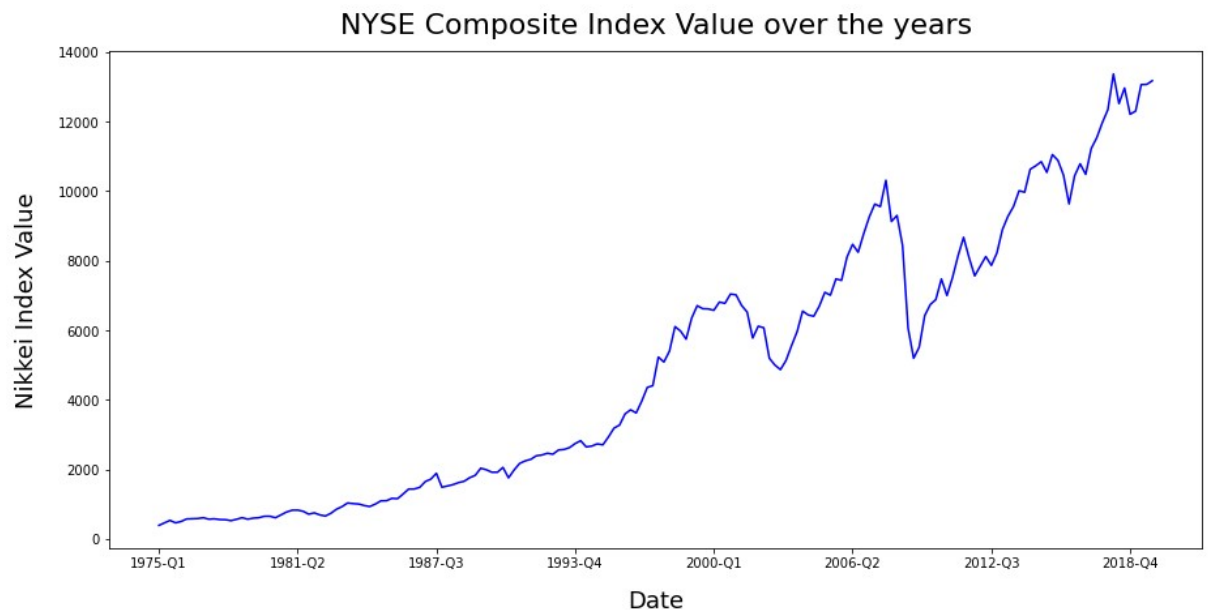


Figure 2.6- NYSE Composite Index Quarterly Index Value (1975-2019)

3. Hypothesis, Methodology, and Variables

3.1 Assumptions underlaying model development

There is one main assumption upon which this exploration has been carried out, the linear relationship between the input and output data. Upon viewing the very similar graphical trends of the equity market indices and Singapore's private real estate index, I deduced that they share a linear corelation, which was the determining factor that led to me choosing a multiple variable **linear** regression model to test K-Fold Cross Validation's real world applicability. The effects of this assumption will be discussed in *Evaluation subsection 5.2*.

3.2 Hypothesis and expected results

In this experiment, four different multiple variable linear regression models will be generated to test K-Fold CV, the first of which will employ simple train-test split to develop the best possible linear regression model (*Model 1.1*), and the rest using K-Fold Cross Validation techniques[12] (*Model 1.2 – Model 1.4*). In addition to accuracy scores, the time complexity of each model will also be tested, as differences in runtime could taint or ameliorate the argument supporting the initial research question. The expected result from this experiment is that Model 1.1 will have the lower R-Squared and Variance Score, with each subsequent model building on and improving the accuracy. However, the time complexity will follow an opposite trend, with Model 1.1 having the shortest runtime and Model 1.4 the longest.

As for the five equity market indices (input data) and their coefficients, it is difficult to determine their order of importance. However, the NYSE composite index encompasses

[12] *Refer to Background Information subsections 2.1.3.1 and 2.1.3.2 for more information on KF techniques*

the greatest number of stocks of all chosen indices[13], so the coefficient assigned to it should be the greatest in value.

3.3 Methodology

3.3.1 Datasets and libraries used in the experiment

Several datasets were used to train the models[14], with daily data from 1975 till the start of 2020 for equity market indices but only one dataset available for private real estate data in Singapore with quarterly data. Due to lack of access to larger datasets for the output data, the private real estate index data was much smaller in size compared to the equity market data- as a result, a new dataset was created, compiling quarterly information for the 5 input indices and the private real estate index to be predicted. This change also made it easier to read, run, and use the data in the experiment.

The entirety of this experiment will be run using Python and its available libraries:

- Dataset formatting will be executing using the *pandas* library.
- Model development and accuracy scoring will use *sklearn* metrics and models.
- The time complexity calculations will be found using the *big_o* library.
- Data plotting will be done using the *matplotlib* library.
- Finally, numerical calculations will be done using *numpy* library.

3.3.2 Procedure

1. Train the model using basic repeated train-test split, varying the size of training and testing groups and repeating the split for each size 1000 times to find the optimal split, **Model 1.1.**

[13] *Refer to Background Information subsections 2.2.2.1 and 2.2.2.5 for Equity market indices description*
[14] *Refer to Bibliography subsection 7.2 for dataset sources*

2. Now, implement the first of the K-Fold methods. Stratified K-Fold only takes one K-value for the dataset type to maintain class spread, so implement that first to obtain **Model 1.2.**
3. Now, apply K-Fold CV with multiple K-values. Record the most accurate split and the K-value which yielded that. This will be **Model 1.3.**
4. Finally, perform Repeated K-Fold CV for an additional 6 times (any more would drastically increase runtime). In the same manner, store the optimum split to obtain **Model 1.4.**

This methodology is subject to technical limitations, such as the lack of access to larger datasets and industry-scale processing power. Time constraints also prevent further repetitions of the Models 1.1 and 1.4.

3.4 Dependent Variables

3.4.1 Accuracy

Accuracy of the individual models is the primary focus of this exploration into K-Fold and Train-Test. The accuracy can be checked by referring to the three previously-mentioned error scoring values- comparing the obtained values indicates the accuracy of each model.

3.4.2 Execution Time[15]

This dependent variable will be measured for each model by calculating time complexity (O[n]) of each model. To do so, the python code for each model has been packed into a function, such that when the *big_o* library is used, the function can be called and it's time complexity calculated.

[15] *Due to differences in CPU performance, exact O(n) values may differ, however the general trend of values will remain the same*

4. Data Collection and Modelling Observed Trends

4.1 Data Collection

The tables below shows the experimental results of running the four different models. Table 1.1 shows the coefficients obtain by each model and Table 1.2 shows the predicted values when the coefficients are applied to the dataset. Finally, Table 1.3 and 1.4 shows the accuracy scoring and time complexity values, which can be used to evaluate the models. A high number of decimal places were used to increase accuracy and differentiate results, which were often very close to one another.

4.1.1 Table 1.1: Coefficient Values

	DJ	S&P 500	NQ	NI 225	NYSE
Model 1.1	0.00428	-0.04549	-0.00259	0.00053	0.01359
Model 1.2	122.91824	-172.9646	-15.11978	15.44370	190.95118
Model 1.3	0.00421	-0.05382	-0.00149	0.00045	0.01492
Model 1.4	0.00422	-0.05463	-0.00145	0.00045	0.01504

4.1.2 Table 1.2: MSE, Variance Scores, and R^2 Values

	Mean Squared Error	Variance Score	R-Squared Values
Model 1.1	747.24488	0.95446	0.60050
Model 1.2	1024.64326	0.93092	0.43850
Model 1.3	725.30573	0.99458	0.61425
Model 1.4	694.20860	0.99773	0.78621

4.1.3 Table 1.3: Time complexities

Due to unavoidable variation in my PC's processing speed, the time complexity calculated varied slightly when I ran the code at different times. The values shown below are the most commonly-repeating time complexities calculated.

	Time Complexity (n = 181) (sec)
Model 1.1	2.7 + 3.3E-07*n
Model 1.2	0.11 + -5E-08*n
Model 1.3	14 + 4.4E-05*n
Model 1.4	44 + 3.5E-05*n

4.2 Visualizing the data

To further aid in the analysis of the data, the values have been plotted in a graphical format to indicate the trends across models.

4.2.1 Figure 3.1 & 3.2: Coefficient Values

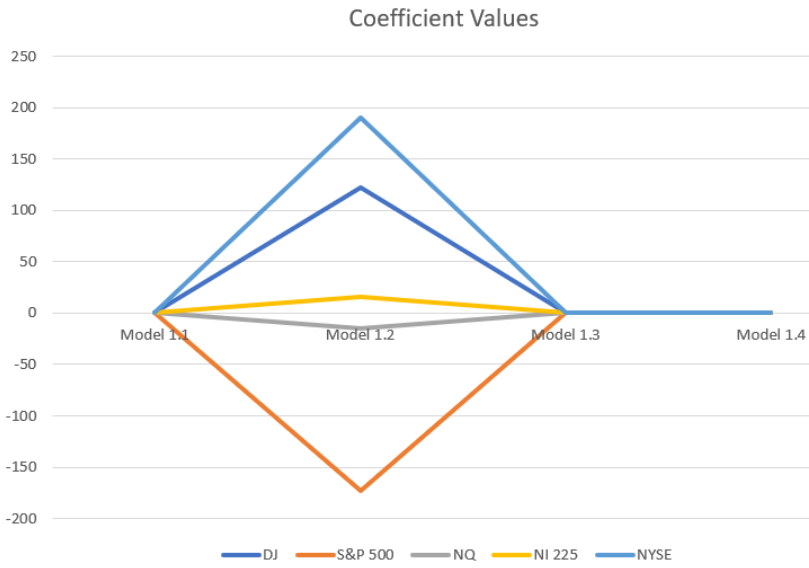


Figure 3.1- Experimental Coefficient Values for the 4 models

The graph shows large disparity in Model 1.2's coefficients which inhibits analysing the other model's values- if this model is removed, we can see small differences in coefficient values across the other models.

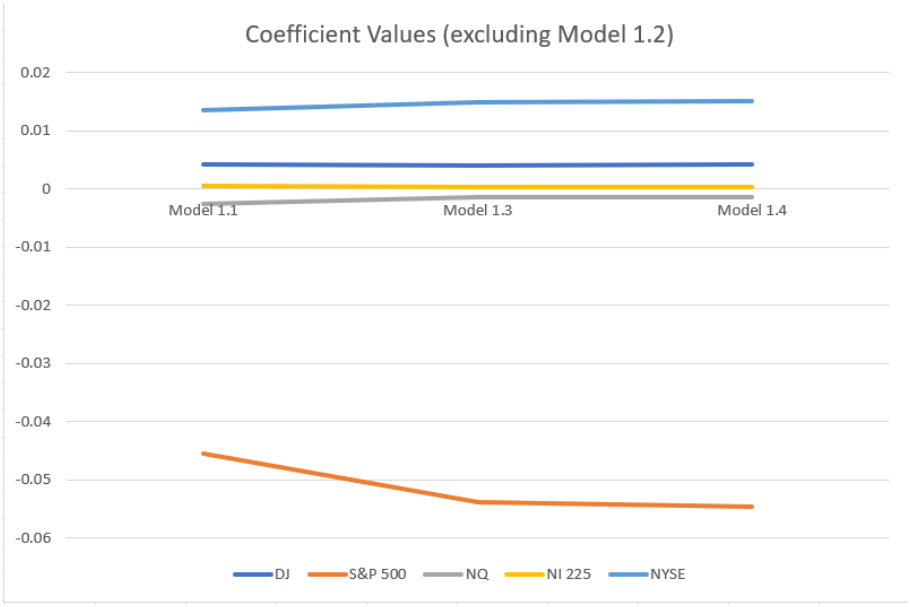


Figure 3.2- Experimental Coefficient Values excluding Model 1.2

4.2.2 Figure 3.3 & 3.4: Variance Score, R2 Values, MSE Score, and Time Complexity

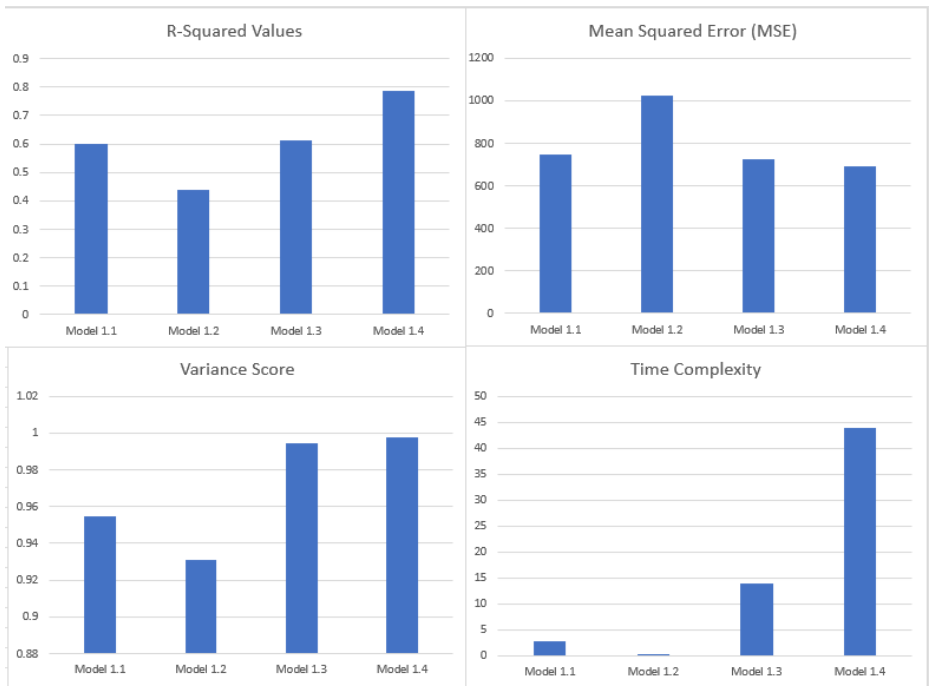


Figure 3.3- Experimental Accuracy Scores and Time Complexites

For the same reasoning as with coefficient values, Figure 3.4 below shows the trends excluding Model 1.2.

Figure 3.4- Experimental Accuracy Scores and Time Complexites excluding Model 1.2

4.2.3 Figure 4.1 – 4.4: Modelling Predicted v True Trends[16]

Table 1.4 contains large amounts of data pertaining to the predicted trends, which has been split up into 4 graphs corresponding to the model used to obtain the predicted trend.

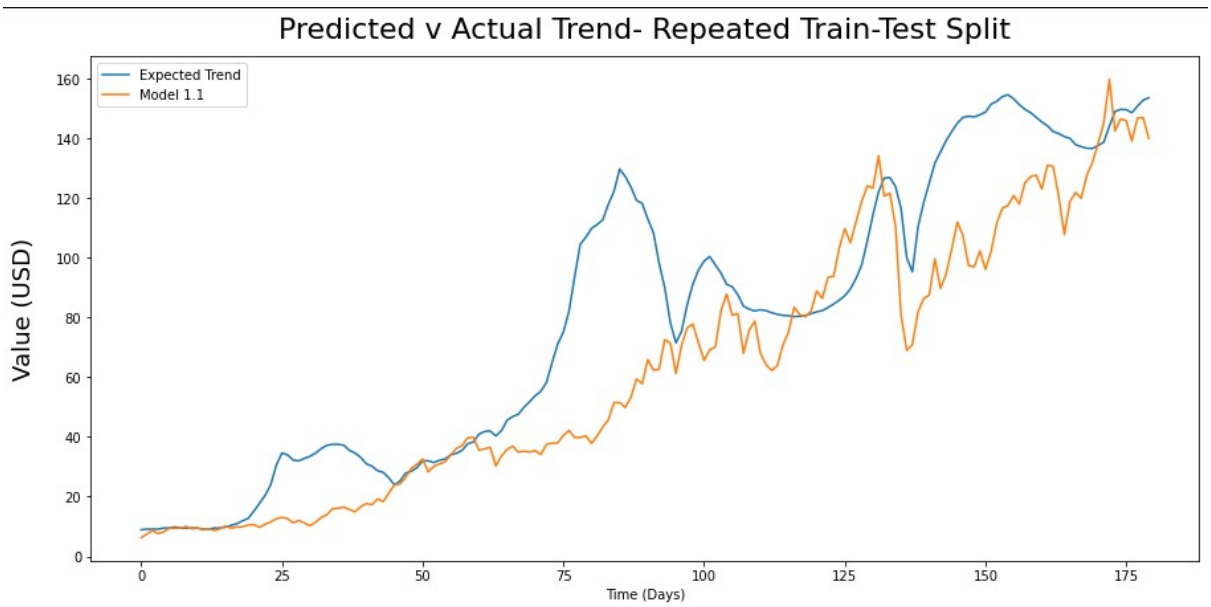


Figure 4.1- Predicted v Actual Trends for Model 1.1

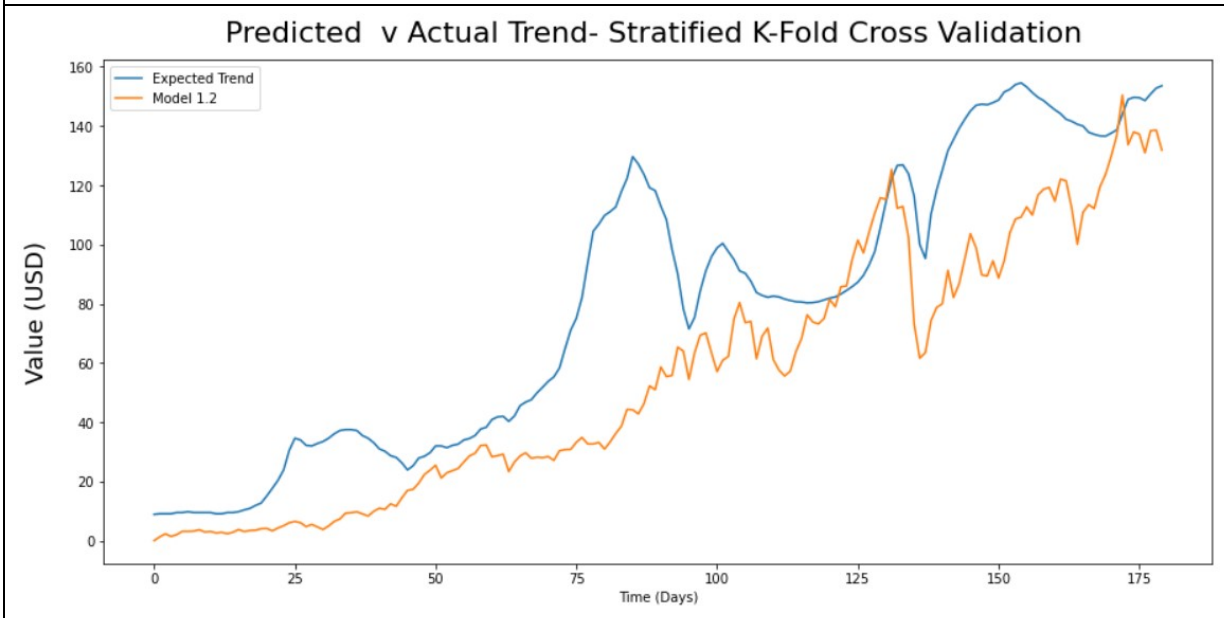


Figure 4.2- Predicted v Actual Trends for Model 1.2

[16] Refer to Appendix subsection 8.2.1.1 for table of values plotted in Figure 4.1 - 4.4 & subsection 8.1.7.1-8.1.7.4 for code sets used to obtain graphs

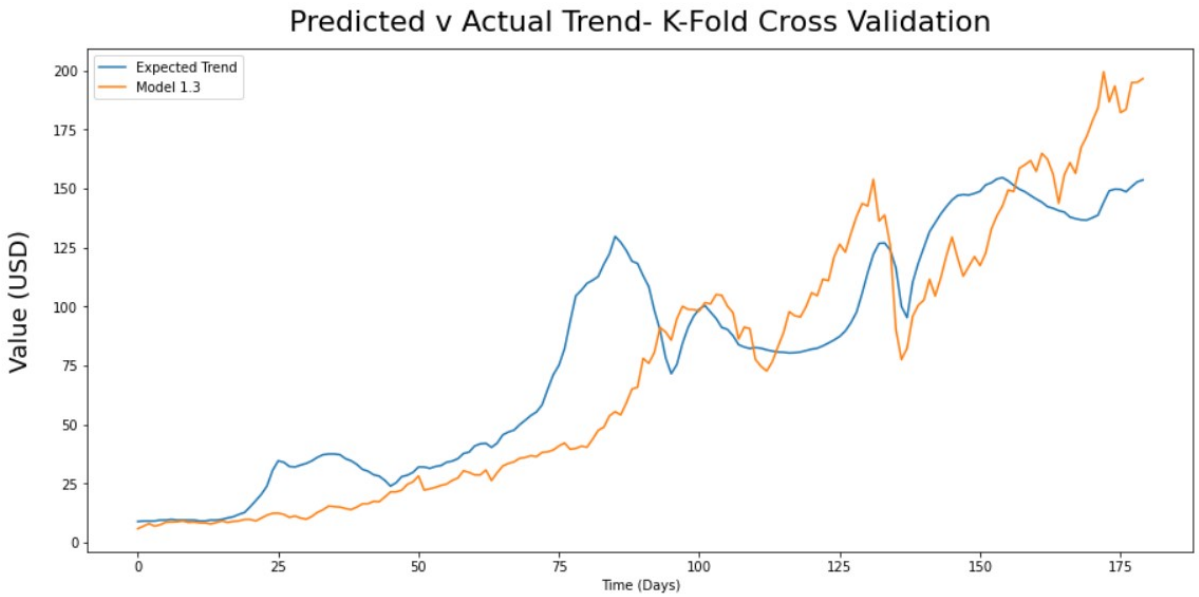


Figure 4.3- Predicted v Actual Trends for Model 1.3

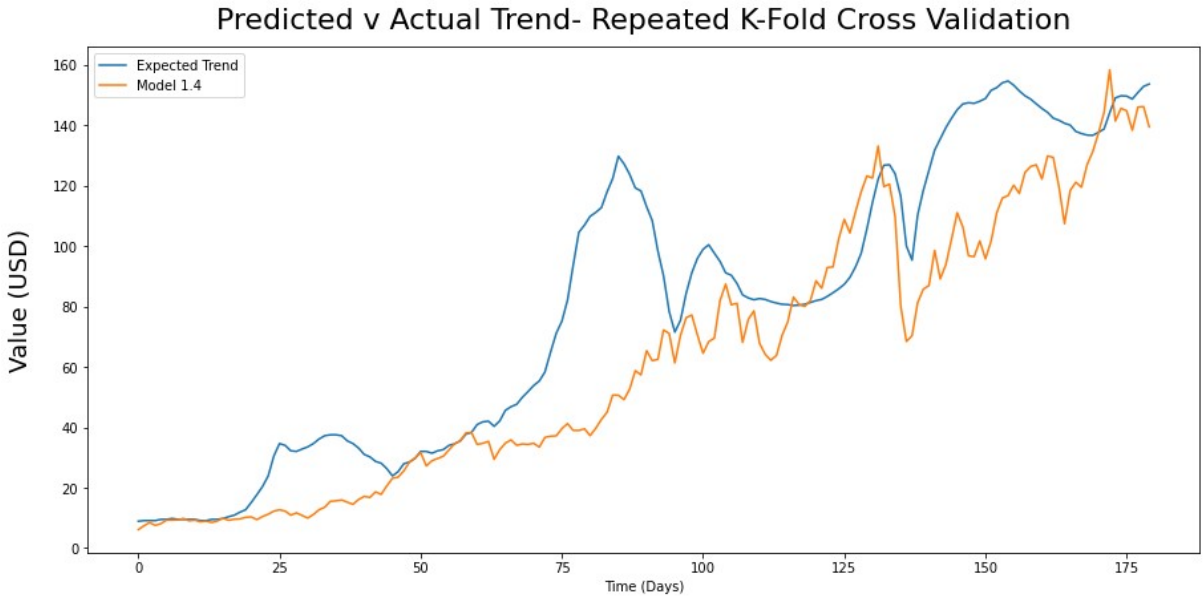


Figure 4.4- Predicted v Actual Trends for Model 1.4

4.3 Data Interpretation[17]

The experimental results largely follow the hypothesized trends for the dependent variables. For accuracy, Models 1.1 through 1.4 progressively got better, with the variance score increasing from 0.95446 to 0.99773 while showing reduced MSE and increased R^2 values. The similar predicted trend was also observed for time complexity,

[17] *Model 1.2 experimentally proved to be erroneous in every category measured. This disparity will be discussed in the evaluation subsection 5.1, and thus model 1.2 will be excluded in the remainder of this section.*

with the repeated train-test split showing much shorter average time complexity than K-Fold and Repeated K-Fold Cross Validation. Figures 4.1-4.4 show that the four models were quite accurate in assigning their coefficients.

5. Evaluation

5.1 Evaluation

In this evaluation section, the results of this experiment will be evaluated with respect to the hypothesized trends and initial research postulate about K-Fold's applicability. Before that, the erroneous data in Model 1.2 needs to be acknowledged. To maintain class spread, only one possible k-value could be used to run the program and train the model (k = 15). This fact definitively explains the disparities observed in the data, from the short 0.01 second runtime, to greatly differing model parameters from the other models (122.91824, -172.9646, -15.11978, 15.44370, 190.95118), to the higher and lower MSE and R^2 values respectively (despite Model 1.2 showing a visually accurate trend). Taking all these discrepancies into account, the results of Model 1.2 will be excluded from the rest of this evaluation section.

5.1.1 Hypothesis Accuracy

The experimental results supported the overarching hypothesis, with the exception of Model 1.2, but nonetheless there were still some unexpected trends observed.

The variance score and R^2 values showed an upward trend in subsequent models. The large increase in variance score (0.04012) from Model 1.1 to 1.3 indicates that switching from simple repeated train-test to K-Fold has a significant impact on the model generated. Furthering this to Repeating K-Fold (Model 1.4) did indeed find an even better fold for the data. Although this reflected poorly in the small increase of variance score (0.00315), the R^2 value showed a greater delta, with the value increasing by a large amount (0.17216) from Model 1.3 to 1.4, as opposed to a smaller delta of 0.0375 from Model *1.1* to *1.3*. This unexpected mirror image in the accuracy scoring

metrics indicates that even the similar-looking trends of Models 1.1, 1.3, and 1.4 differed greatly in terms of their ESS, TSS, and Variance[18].

As for time complexities, they concurred to theoretical predictions. Given the inherent nature Model 1.1, it had the smallest time complexity (≈2.7s), more than five times less than Model 1.2 (≈14s). Despite train-test splitting being performed 1000 times, the training process was much simpler- for K-Fold, the data was split into *k* splits and each split checked, which increased non-linearly as the *k* values got bigger and bigger. Expressing this mathematically, we get:

$$O((n \times 1) + (n \times 2) + \dots (n \times K)), \text{ for sample size } n \text{ and } K \text{ splits}$$

$$\Rightarrow \mathbf{O}\left(\sum_{i=1}^{n}(n \times i)\right),$$

To repeat this process an additional 6 times would certainly only increase the time complexity further, which was seen in the last model (≈44s).

Finally, there was nothing noteworthy observed concerning the coefficient predictions- across all models, including the erroneous one, NYSE Composite was assigned the largest coefficient, indicating that it held the greatest sway over predicted values.

5.1.2 Applicability Analysis

By and large, the overlap between the experimental results and hypothesized trends supports the argument that K-Fold Cross Validation techniques can be used to develop an accurate private real estate price predictor model. The increase in variance score from Model 1.1 (0.95446) to Model 1.4 (0.99773) supports the claim that K-Fold Cross Validation is indeed more accurate than simply repeating Train-Test Splitting. In addition to this, a soft factor that doesn't show up in accuracy scoring rubrics is K-

[18] *Refer to Background Information subsection 2.1.5.2 for ESS and TSS full descriptions*

Fold's ability to remove model overfitting, a situation where a generated model is overly tailored to a limited set of points, often the case for train-test models like3 1.1. By training on several folds, the model does not become extremely tailored to the individual dataset.

High accuracy and no overfitting? It that K-Fold's scope for applicability in the real world to train Machine Learning models is evident. For this nuanced case, it's high accuracy can allow investors to make the best-possible informed decisions concerning buying and selling of Singaporean real estate.

However, it would be dogmatic to look at variance score as the sole lens of justification spotlighting K-Fold CV. The larger time complexity associated with K-Fold necessitates much more processing power and time when compared to the faster Train-Test Split, which taints the argument that K-Fold Cross Validation should be applied in real world situations. The marginal increase in variance score from Model 1.3 to 1.4 (0.000315) was accompanied by a 30-second increase in time complexity, and around 41 seconds when compared to Model 1.1. When such models are fitted with industry-scale data in the finance sector or any real-life situation, the time complexity difference will only increase exponentially, dramatically reducing K-Fold Cross Validation's applicability in the modern digital world.

5.2 Experimentation Limitations

While the experiment can be viewed at as successful, there were certain limitations in this experiment that rendered it a sub-optimal modelling of the real world and an imperfect analysis of K-Fold Cross Validation's real world applicability analysis.

Firstly, the relation between the equity market indices and Singapore's private real estate price index was indirectly assumed to be a linear one, since the two techniques were used

on a multivariate *linear* regression model. In the real world, a logarithmic or exponential relation could exist between the input data and value to be predicted which may affect the results obtained.

Secondly, the experiment was performed by an IB Diploma student, not an industry professional with access to industrial-scale datasets and compute power. Around 200 rows of input data were used in this experiment, which is satisfactory to reach a decisive conclusion, but the presence of more data would have further improved the accuracy of all four models.

5.3 Future research scopes

To further investigate the correctness of the hypothesis and even the obtained values, there are several areas where this essay can be expanded.

The first and most important future expansion would be to apply K-Fold Cross Validation techniques on a non-linear regression model, thus testing whether certain market indices have a non-linear relationship with the private real estate index. This extension would overcome one of the previously-mentioned unrealistic assumption of this exploration, the assumption of linearity between the inputs and output.

As is the case with most machine learning models, using more data would increase the accuracy of this exploration, whether that means considering more data for the same indices or using more equity market indices to develop a more precise model, given that there is adequate processing power to work on the large volumes data efficiently. This increased processing power would also permit increased repetitions for Models 1.1 and 1.4, further boosting their respective accuracies.

6. Conclusion

This essay aimed to explore K-Fold Cross Validation from the lens of accuracy and runtime when applied in the development of multiple variable linear regression models, with the particular case study of predicting Singapore's private real estate price index using five important equity market indices (Dow Jones, S&P 500, NASDAQ, Nikkei 225, and NYSE Composite). The primary goal of this exploration was sufficiently met, with the results obtained from the experiment supported the initial overarching hypotheses. However, this essay also yielded experimental values which allow for differing interpretations of K-Fold and its related techniques. From one lens of view, K-Fold is a clear candidate, boasting higher variance scores and thus greater accuracy in value prediction, whilst from another, K-Fold has a large time complexity necessitating greater processing power and time to execute a task. Real Estate Index prediction is just one situation, the pros and cons of the technique need to be weighed before deciding whether to implement it in a real life situation

Nevertheless, this essay build a sufficient argument for the implementation of such machine learning models in real life situations. Even the most inaccurate of the four models (0.93092 variance score) showed high levels of accuracy in predicting Singapore's Private Real Estate Index with only quarterly equity market information from 1975 onwards, with the graphical trends being nearly identical. Thus, this exploration serves as a clear indicator of the power of machine learning, justifying their applicability in the financial sector and beyond.

7. Bibliography

7.1 Sources used

- Tagliaferri, L. (2021, December 29). *An Introduction to Machine Learning*. DigitalOcean. Retrieved January 1, 2921, from https://www.digitalocean.com/community/tutorials/an-introduction-to-machine-learning
- Fumo, D. (2018, June 21). *Types of Machine Learning Algorithms You Should Know*. Medium. Retrieved July 28, 2021, from https://towardsdatascience.com/types-of-machine-learning-algorithms-you-should-know-953a08248861
- Heidenreich, H. (2021, December 7). *What are the types of machine learning? - Towards Data Science*. Medium. Retrieved July 28, 2021, from https://towardsdatascience.com/what-are-the-types-of-machine-learning-e2b9e5d1756f
- *Market Index*. (2021, May 30). Investopedia. Retrieved July 29, 2021, from https://www.investopedia.com/terms/m/marketindex.asp
- *The Housing Price Index Explained - Discover*. (2020). House Price Index: What Does It Mean to a Home Buyer? Retrieved July 29, 2021, from https://www.discover.com/home-loans/articles/house-price-index-what-does-it-mean-to-a-home-buyer/
- *Holdings*. (2021, February 15). Investopedia. Retrieved July 29, 2021, from https://www.investopedia.com/terms/h/holdings.asp
- *What Is the Dow Jones Industrial Average (DJIA)?* (2022, January 3). Investopedia. Retrieved July 30, 2021, from https://www.investopedia.com/terms/d/djia.asp
- *What is a Blue Chip?* (2020, December 27). Investopedia. Retrieved July 30, 2021, from https://www.investopedia.com/terms/b/bluechip.asp
- *The S&P 500 Index*. (2021, December 22). Investopedia. Retrieved July 30, 2021, from https://www.investopedia.com/terms/s/sp500.asp

- *What Is the Nasdaq Composite Index?* (2022, January 6). Investopedia. Retrieved July 30, 2021, from https://www.investopedia.com/terms/n/nasdaqcompositeindex.asp
- *What Is Nasdaq?* (2021, November 21). Investopedia. Retrieved July 30, 2021, from https://www.investopedia.com/terms/n/nasdaq.asp
- Wikipedia contributors. (2022, January 8). *Nikkei 225*. Wikipedia. Retrieved July 30, 2021, from https://en.wikipedia.org/wiki/Nikkei_225
- *NYSE Composite Index*. (2020, November 15). Investopedia. Retrieved July 30, 2021, from https://www.investopedia.com/terms/n/nysecompositeindex.asp
- *Code Faster with Line-of-Code Completions, Cloudless Processing*. (2019). How to Add a Legend to a Matplotlib Plot in Python. Retrieved July 30, 2021, from https://www.kite.com/python/answers/how-to-add-a-legend-to-a-matplotlib-plot-in-python
- Rowe, W. (2018, July 5). *Mean Square Error & R2 Score Clearly Explained*. BMC Blogs. Retrieved July 30, 2021, from https://www.bmc.com/blogs/mean-squared-error-r2-and-variance-in-regression-analysis/
- Brownlee, J. (2020, August 26). *Repeated k-Fold Cross-Validation for Model Evaluation in Python*. Machine Learning Mastery. Retrieved October 1, 2021, from https://machinelearningmastery.com/repeated-k-fold-cross-validation-with-python/
- Robinson, S. (2021, June 7). *Linear Regression in Python with Scikit-Learn*. Stack Abuse. Retrieved October 2, 2021, from https://stackabuse.com/linear-regression-in-python-with-scikit-learn/
- *Big Data: What it is and why it matters*. (2021). SAS. Retrieved January 6, 2022, from https://www.sas.com/en_sg/insights/big-data/what-is-big-data.html

- *How Much Data Is Created Every Day? [27 Staggering Stats]*. (2021, December 4). SeedScientific. Retrieved January 6, 2022, from https://seedscientific.com/how-much-data-is-created-every-day/
- GeeksforGeeks. (2021, May 29). *Stratified K Fold Cross Validation*. Retrieved January 8, 2022, from https://www.geeksforgeeks.org/stratified-k-fold-cross-validation/
- S. (2020, December 28). *Absolute Error & Mean Absolute Error (MAE)*. Statistics How To. Retrieved January 8, 2022, from https://www.statisticshowto.com/absolute-error/
- Brownlee, J. (2020, August 26). *Train-Test Split for Evaluating Machine Learning Algorithms*. Machine Learning Mastery. Retrieved January 9, 2022, from https://machinelearningmastery.com/train-test-split-for-evaluating-machine-learning-algorithms/
- Wikipedia contributors. (2022, January 9). *Cross-validation (statistics)*. Wikipedia. Retrieved January 10, 2022, from https://en.wikipedia.org/wiki/Cross-validation_(statistics)
- Alto, V. (2021, December 11). *Simple Linear Regression with Python - DataDrivenInvestor*. Medium. Retrieved January 10, 2022, from https://medium.datadriveninvestor.com/simple-linear-regression-with-python-1b028386e5cd
- *Understanding stratified cross-validation*. (2013, February 7). Cross Validated. Retrieved January 12, 2022, from https://stats.stackexchange.com/questions/49540/understanding-stratified-cross-validation
- Alto, V. (2021b, December 11). *Understanding the OLS method for Simple Linear Regression*. Medium. Retrieved January 12, 2022, from https://towardsdatascience.com/understanding-the-ols-method-for-simple-linear-regression-e0a4e8f692cc

- Poston, D. (2022). *Ordinary Least Squares Regression | Encyclopedia.com*. Ordinary Least Squares Regression. Retrieved January 12, 2022, from https://www.encyclopedia.com/social-sciences/applied-and-social-sciences-magazines/ordinary-least-squares-regression
- Alto, V. (2021b, December 11). *Simple Linear Regression with Python - DataDrivenInvestor*. Medium. Retrieved January 12, 2022, from https://medium.datadriveninvestor.com/simple-linear-regression-with-python-1b028386e5cd

7.2 Dataset Sources

- Urban Redevelopment Authority. (1975–2020, January 1–March 31). *Private Residential Property Price Index* [To provide quarterly data on price indices of private properties]. Data.gov.sg. https://data.gov.sg/dataset/private-residential-property-price-index-by-type-of-property
- Yahoo! Finance. (1975–2021, January 1–March 31). *Dow Jones Industrial Average (^DJI)* [DJI Real Time Price. Currency in USD]. Yahoo! Finance. https://finance.yahoo.com/quote/%5EDJI/history/
- Yahoo! Finance. (1975–2020, January 1–March 31). *S&P 500 (^GSPC)* [SNP Real Time Price. Currency in USD]. Yahoo! Finance. https://finance.yahoo.com/quote/%5EGSPC/history/
- Yahoo! Finance. (1975–2020a, January 1–March 31). *NASDAQ Composite (^IXIC)* [Nasdaq GIDS Real Time Price. Currency in USD]. Yahoo! Finance. https://finance.yahoo.com/quote/%5EIXIC/history/
- Yahoo! Finance. (1975–2020b, January 1–March 31). *Nikkei 225 (^N225)* [Osaka Delayed Price. Currency in JPY]. Yahoo! Finance. https://finance.yahoo.com/quote/%5EN225/history/

- Yahoo! Finance. (1975–2020c, January 1–March 31). *NYSE COMPOSITE (DJ) (^NYA)* [NYSE Delayed Price. Currency in USD]. Yahoo! Finance. https://finance.yahoo.com/quote/%5ENYA/history/

8. Appendix

8.1 Python Code used to obtain experimental observations

8.1.1 Importing necessary libraries for the code sets

```
import pandas as pd

import pylab as pl

import big_o

from statistics import mean


import matplotlib.pyplot as plt

from matplotlib import legend, figure


import numpy as np

from numpy import mean

from numpy import std

from numpy import absolute


from sklearn import linear_model

from sklearn import preprocessing


from sklearn.metrics import mean_squared_error

from sklearn.metrics import f1_score

from sklearn.metrics import r2_score


from sklearn.model_selection import KFold

from sklearn.model_selection import RepeatedKFold
```

```
from sklearn.model_selection import StratifiedKFold
from sklearn.model_selection import train_test_split
from sklearn.model_selection import cross_val_score

from sklearn.linear_model import Lasso

%matplotlib inline
```

8.1.2 Code set used to read and plot raw data in tabular and graph form

Note- A similar code set has been repeated for each equity market index to obtain the trend. However, all instances of code implementation have not been attached in the appendix, as they were quite similar in nature and the code attached below is sufficiently representative of them.

```
PrivData =
pd.read_csv(r'C:\Users\rohan\OneDrive\Rohan\IB\Core\EE
[Extended Essay]\Final Extended Essay\Datasets\Final
Datasets\PrivResData.csv')

PRV = PrivData[['Quarter_PRD', 'Value_PRD']]
raw_data = pd.DataFrame({'year & quarter':
PRV.Quarter_PRD, 'value': PRV.Value_PRD})
raw_data.set_index('year &
quarter')['value'].plot(figsize=(15, 7), linewidth=1.5,
color='blue')

plt.xlabel("Year & Quarter", labelpad=15, fontsize = 18)
```

```
plt.ylabel("Index Value", labelpad=15, fontsize = 18)
plt.title("Private Real Estate Price Index Value [1975-
2019]", y=1.02, fontsize=22)
```

8.1.3 Generalizing Datasets and Initializing Lists

```
SplitLens = [0.5, 0.6, 0.7, 0.8, 0.9]
SplitName = ['50-50 Split', '60-40 Split', '70-30 Split',
'80-20 Split', '90-10 Split']
raw_dataset =
pd.read_csv(r'C:\Users\rohan\OneDrive\Rohan\IB\Core\EE
[Extended Essay]\Final Extended Essay\Datasets\Final
Datasets\Combined Data Set.csv')
ML_data = raw_dataset[['Price_DJ' , 'Price_SP500' ,
'Price_NQ', 'Price_NI' , 'Price_NY' , 'Value_PRD']]
X = np.asanyarray(raw_dataset[['Price_DJ' , 'Price_SP500'
, 'Price_NQ', 'Price_NI' , 'Price_NY']])
Y_true = np.asanyarray(raw_dataset[['Value_PRD']])
Y_True = []
for i in Y_true:
    Y_True.append(i[0])
```

8.1.4 Time Complexity Calculation

Note- For subsections 8.1.4 through 8.1.6, a similar code set has been repeated for each model. However, all instances of code implementation have not been attached in the appendix, as they were quite similar in nature and the code attached below is sufficiently representative of them.

```
sample_strings = lambda n: big_o.datagen.strings(181)
```

```
best, others = big_o.big_o(Model_1_1, sample_strings,
n_measures=10)
for class_, residuals in others.items():
    print(class_)
```

8.1.5 Tabulating Predicted v True Values

```
Table = {'Model 1.4 Predicted: ': Y_Pred_SKF, 'True: ':
Y_True}
df = pd.DataFrame(Table)
pd.set_option("display.max_rows", None,
"display.max_columns", None)
df.head(181)
```

8.1.6 Accuracy Scoring

```
print("R-Squared Score:", r2_score(Y_Pred_TTS, Y_True))
print("Variance score:", max_score_TTS)
print("Mean Squared Error (MSE):",
mean_squared_error(Y_Pred_TTS, Y_True, squared = True))
```

8.1.7 Code set used in the experiment to generate the models

8.1.7.1 Model 1.1- Repeated Train-Test Split Model

```
def Model_1_1(val):


    global Optimal_TTS_Coefs, Optimal_TTS_Split,
Optimal_Score_TTS, Optimal_Split_TTS
    Optimal_TTS_Coefs = []
    Optimal_TTS_Split = []
```

```
    Optimal_Score_TTS = 0
    Optimal_Split_TTS = 0


    for x in range(1000):
        for i in range(len(SplitLens)):
            split = np.random.rand(len(raw_dataset)) <
SplitLens[i]
            training_set_TTS = ML_data[split]
            testing_set_TTS = ML_data[~split]


            Model_TTS = linear_model.LinearRegression()
            Training_Input_TTS =
np.asanyarray(training_set_TTS[['Price_DJ' ,
'Price_SP500' , 'Price_NQ', 'Price_NI' , 'Price_NY']])
            Training_Output_TTS =
np.asanyarray(training_set_TTS[['Value_PRD']])
            Model_TTS.fit (Training_Input_TTS,
Training_Output_TTS)


            y_hat=
Model_TTS.predict(testing_set_TTS[['Price_DJ' ,
'Price_SP500' , 'Price_NQ', 'Price_NI' , 'Price_NY']])
            Testing_Input_TTS =
np.asanyarray(testing_set_TTS[['Price_DJ' ,
'Price_SP500' , 'Price_NQ', 'Price_NI' , 'Price_NY']])
```

```
            Testing_Output_TTS =
np.asanyarray(testing_set_TTS[['Value_PRD']])
            score_TTS =
Model_TTS.score(Testing_Input_TTS, Testing_Output_TTS)
            if score_TTS > Optimal_Score_TTS:
                Optimal_Score_TTS = score_TTS
                Optimal_Split_TTS = i
                Optimal_TTS_Coefs = Model_TTS.coef_
                Optimal_TTS_Split = split

    training_set_TTS = ML_data[Optimal_TTS_Split]
    testing_set_TTS = ML_data[~Optimal_TTS_Split]

    Training_Input_TTS =
np.asanyarray(training_set_TTS[['Price_DJ' ,
'Price_SP500' , 'Price_NQ', 'Price_NI' , 'Price_NY']])
    Training_Output_TTS =
np.asanyarray(training_set_TTS[['Value_PRD']])
    Testing_Input_TTS =
np.asanyarray(testing_set_TTS[['Price_DJ' ,
'Price_SP500' , 'Price_NQ', 'Price_NI' , 'Price_NY']])
    Testing_Output_TTS =
np.asanyarray(testing_set_TTS[['Value_PRD']])

    Model_TTS.fit (Training_Input_TTS,
Training_Output_TTS)
```

```
    y_hat=
Model_TTS.predict(testing_set_TTS[['Price_DJ' ,
'Price_SP500' , 'Price_NQ', 'Price_NI' , 'Price_NY']])
    score = Model_TTS.score(Testing_Input_TTS,
Testing_Output_TTS)
    print("Optimal Coefficients:", Optimal_TTS_Coefs)
    print("Optimal Split:",
SplitName[Optimal_Split_TTS])
    print( "Optimal Variance Score:", score)
Model_1_1(1)
```

8.1.7.2 Model 1.2- Stratified K-Fold Cross Validation

```
scaler = preprocessing.MinMaxScaler()
X_scaled = scaler.fit_transform(X)
Y_true_int = []
for i in Y_true:
    Y_true_int.append(int(i))
def Model_1_2(val):
    global SKFold_model, max_score_SKF,
max_score_split_SKF
    x_train_SKF = []
    x_test_SKF = []
    y_train_SKF = []
    y_test_SKF = []
    max_score_SKF = 0
    max_score_split_SKF = []
```

```
    optimum_k = 0


    for k in range (2,16):
        SKFsplit = StratifiedKFold(n_splits=k, shuffle
= False, random_state = None)
        for train, test in
SKFsplit.split(X_scaled,Y_true_int):
            for i in train:
                x_train_SKF.append(X_scaled[i])
                y_train_SKF.append(Y_true_int[i])
            for j in test:
                x_test_SKF.append(X_scaled[j])
                y_test_SKF.append(Y_true_int[j])
            SKFold_model =
linear_model.LinearRegression()
            SKFold_model.fit (x_train_SKF, y_train_SKF)
            SKFold_y_hat_SKF=
SKFold_model.predict(x_test_SKF)
            score = SKFold_model.score(x_test_SKF,
y_test_SKF)
            if score > max_score_SKF:
                optimum_k_SKF = k
                max_score_SKF = score
                max_score_split_SKF = []
                max_score_split_SKF.append(train)
                max_score_split_SKF.append(test)
```

```
            x_train_SKF = []
            x_test_SKF = []
            y_train_SKF = []
            y_test_SKF = []

    for i in (max_score_split_SKF[0]):
        x_train_SKF.append(X_scaled[i])
        y_train_SKF.append(Y_true_int[i])
    for j in (max_score_split_SKF[1]):
        x_test_SKF.append(X_scaled[j])
        y_test_SKF.append(Y_true_int[j])

    print("Optimum 'k' value:", optimum_k_SKF)
    SKFold_model = linear_model.LinearRegression()
    SKFold_model.fit (x_train_SKF, y_train_SKF)
    print ('Coefficients for maximum variance: ',
SKFold_model.coef_)
    SKFold_y_hat= SKFold_model.predict(x_test_SKF)
    print("Minimum variance score:", max_score_SKF)
Model_1_2(1)
Coefficients_SKF = (SKFold_model.coef_)
print(Coefficients_SKF)
Y_Pred_SKF = []

for j in range(len(X_scaled)):
    total = 0
```

```
    for k in range (0,5):
        total = total + ((Coefficients_SKF[k]) *
((X_scaled[j])[k]))
    Y_Pred_SKF.append(total)
```

8.1.7.3 Model 1.3- K-Fold Cross Validation

```
def Model_1_3(val):
    global max_score_KF, max_score_split_KF,
optimum_k_KF, KFold_model
    x_train_KF = []
    x_test_KF = []
    y_train_KF = []
    y_test_KF = []
    max_score_KF = 0
    max_score_split_KF = []
    optimum_k_KF = 0

    for k in range (2,91):
        KFsplit = KFold(n_splits=k, shuffle = True,
random_state = 69)
        for train, test in KFsplit.split(X):
            for i in train:
                x_train_KF.append(X[i])
                y_train_KF.append(Y_true[i])
            for j in test:
                x_test_KF.append(X[j])
```

```
                y_test_KF.append(Y_true[j])
            KFold_model =
linear_model.LinearRegression()
            KFold_model.fit (x_train_KF, y_train_KF)
            KFold_y_hat= KFold_model.predict(x_test_KF)
            score_KF = KFold_model.score(x_test_KF,
y_test_KF)
            if score_KF > max_score_KF:
                optimum_k_KF = k
                max_score_KF = score_KF
                max_score_split_KF = []
                max_score_split_KF.append(train)
                max_score_split_KF.append(test)
            x_train_KF = []
            x_test_KF = []
            y_train_KF = []
            y_test_KF = []

    for i in (max_score_split_KF[0]):
        x_train_KF.append(X[i])
        y_train_KF.append(Y_true[i])
    for j in (max_score_split_KF[1]):
        x_test_KF.append(X[j])
        y_test_KF.append(Y_true[j])

    print("Optimum 'k' value:", optimum_k_KF)
```

```
    KFold_model = linear_model.LinearRegression()
    KFold_model.fit (x_train_KF, y_train_KF)
    print ('Coefficients for maximum variance: ',
KFold_model.coef_)
    KFold_y_hat= KFold_model.predict(x_test_KF)
    print("Maximum variance score:", max_score_KF)
Model_1_3(1)
Coefficients_KF = (KFold_model.coef_)[0]
Y_Pred_KF = []

for j in range(len(X)):
    for k in range (0,5):
        total = 0
        total = total + ((Coefficients_KF[k]) *
((X[j])[k]))
    Y_Pred_KF.append(total)
```

8.1.7.4 Model 1.4- Repeated K-Fold Cross Validation

```
def Model_1_4(val):
    global RKFold_model
    x_train_RKF = []
    x_test_RKF = []
    y_train_RKF = []
    y_test_RKF = []
    max_score_RKF = 0
    max_score_split_RKF = []
```

```
    optimum_k_RKF = 0
    optimum_r = 0


    for r in range (1,6):
        for k in range (2,91):
            RKFsplit = RepeatedKFold(n_splits=k,
n_repeats=r, random_state = 69)
            for train, test in RKFsplit.split(X):
                for i in train:
                    x_train_RKF.append(X[i])
                    y_train_RKF.append(Y_true[i])
                for j in test:
                    x_test_RKF.append(X[j])
                    y_test_RKF.append(Y_true[j])
                RKFold_model =
linear_model.LinearRegression()
                RKFold_model.fit (x_train_RKF,
y_train_RKF)
                RKFold_y_hat=
KFold_model.predict(x_test_RKF)
                score = RKFold_model.score(x_test_RKF,
y_test_RKF)
                if score > max_score_RKF:
                    optimum_k_RKF = k
                    optimum_r = r
                    max_score_RKF = score
```

```
                    max_score_split_RKF = []
                    max_score_split_RKF.append(train)
                    max_score_split_RKF.append(test)
                x_train_RKF = []
                x_test_RKF = []
                y_train_RKF = []
                y_test_RKF = []

            for i in (max_score_split_RKF[0]):
                x_train_RKF.append(X[i])
                y_train_RKF.append(Y_true[i])
            for j in (max_score_split_RKF[1]):
                x_test_RKF.append(X[j])
                y_test_RKF.append(Y_true[j])

    print("Optimum 'k' value:", optimum_k_RKF)
    print("Optimum number of repeats:", optimum_r)
    RKFold_model = linear_model.LinearRegression()
    RKFold_model.fit (x_train_RKF, y_train_RKF)
    print ('Coefficients for maximum variance: ',
RKFold_model.coef_)
    RKFold_y_hat= RKFold_model.predict(x_test_KF)
    print("Minimum variance score:", max_score_KF)
Model_1_4(1)
Coefficients_KF = (KFold_model.coef_)[0]
Y_Pred_KF = []
```

```
for j in range(len(X)):
    total = 0
    for k in range (0,5):
        total = total + ((Coefficients_KF[k]) *
((X[j])[k]))
    Y_Pred_KF.append(total)
```

8.2 Datasets Used

8.2.1 Raw Data Generated

8.2.1.1 Singapore Private Real Estate Index True Values

True Values	
	Private Real Estate Index Values
0	8.9
1	9.1
2	9.1
3	9.1
4	9.5
5	9.5
6	9.8
7	9.5
8	9.5
9	9.5
10	9.5
11	9.1
12	9.1
13	9.5
14	9.5
15	9.8

16	10.4
17	10.9
18	11.9
19	12.7
20	15.1
21	17.7
22	20.4
23	23.9
24	30.5
25	34.6
26	34
27	32.2
28	32
29	32.8
30	33.5
31	34.6
32	36.1
33	37.2
34	37.5
35	37.5
36	37.2
37	35.5
38	34.6
39	33.1
40	31
41	30.2
42	28.7
43	28.1
44	26.3
45	23.9
46	25.3
47	27.9

48	28.5
49	29.7
50	32
51	32
52	31.4
53	32.2
54	32.6
55	34
56	34.5
57	35.5
58	37.7
59	38.3
60	40.9
61	41.8
62	42
63	40.3
64	42.1
65	45.6
66	46.8
67	47.6
68	49.9
69	51.8
70	53.8
71	55.3
72	58.3
73	64.9
74	71.1
75	75.1
76	82.1
77	93.6
78	104.5
79	106.9

80	109.8
81	111.1
82	112.7
83	117.9
84	122.3
85	129.7
86	127.2
87	123.7
88	119.2
89	118.2
90	113
91	108.4
92	98.3
93	90.1
94	78.3
95	71.5
96	75.4
97	84.3
98	91.1
99	95.9
100	98.9
101	100.4
102	97.6
103	94.9
104	91.1
105	90.3
106	87.6
107	83.8
108	82.8
109	82.2
110	82.6
111	82.3

112	81.6
113	81.1
114	80.7
115	80.6
116	80.3
117	80.4
118	80.7
119	81.3
120	81.9
121	82.3
122	83.3
123	84.5
124	85.8
125	87.3
126	89.6
127	93.1
128	97.6
129	105.6
130	114.4
131	122.1
132	126.7
133	126.9
134	123.9
135	116.4
136	100
137	95.3
138	110.3
139	118.4
140	125.1
141	131.7
142	135.5
143	139.2

144	142.3
145	145.1
146	147
147	147.4
148	147.2
149	147.9
150	148.8
151	151.5
152	152.4
153	154
154	154.6
155	153.2
156	151.3
157	149.7
158	148.6
159	147
160	145.5
161	144.2
162	142.3
163	141.6
164	140.6
165	140
166	137.9
167	137.2
168	136.7
169	136.6
170	137.6
171	138.7
172	144.1
173	149
174	149.7
175	149.6

176	148.6
177	150.8
178	152.8
179	153.6

8.2.1.2 Predicted Values Generated per Model

Predicted Values				
	Model 1.1	**Model 1.2**	**Model 1.3**	**Model 1.4**
0	5.950872	5.846838	0.045525	6.133424
1	7.180225	6.916505	1.301354	7.384693
2	8.322055	7.995722	2.323188	8.449773
3	7.326555	6.935455	1.429334	7.516478
4	7.874442	7.516020	2.021831	8.096174
5	9.051466	8.604637	3.158524	9.252056
6	9.046246	8.726092	3.145132	9.245120
7	9.117172	8.787715	3.221395	9.319550
8	9.653780	9.131637	3.689407	9.808492
9	8.857408	8.510039	2.923143	9.032055
10	9.089687	8.664618	3.120972	9.242239
11	8.587727	8.331737	2.610667	8.734206
12	8.820167	8.282797	2.817097	8.945168
13	8.324053	7.864717	2.367246	8.477431
14	8.819834	8.465874	2.861118	8.977046
15	9.770444	9.147304	3.763151	9.896695
16	9.039486	8.508398	3.083427	9.196076
17	9.417542	8.950201	3.435998	9.561609
18	9.501655	9.128504	3.537392	9.660268
19	10.058958	9.756368	4.045771	10.190445
20	10.183747	9.773676	4.158022	10.307739
21	9.267164	9.123729	3.315394	9.440474
22	10.285808	10.370058	4.300241	10.445954
23	11.067456	11.575354	5.050973	11.222790

24	12.120018	12.346906	6.072137	12.251087
25	12.565516	12.403753	6.495437	12.673886
26	12.172995	11.890928	6.070893	12.257842
27	10.763313	10.649374	4.742956	10.899138
28	11.581508	11.218899	5.493808	11.670393
29	10.707190	10.332159	4.654747	10.814229
30	9.785938	9.862007	3.790659	9.934985
31	10.941778	11.045370	4.975602	11.116010
32	12.546395	12.783933	6.523949	12.692072
33	13.394851	13.888366	7.347451	13.530788
34	15.382251	15.446986	9.294949	15.496247
35	15.498103	15.183635	9.461369	15.640139
36	15.773465	15.016373	9.763243	15.932936
37	15.038159	14.361652	9.027500	15.193977
38	14.248709	13.935665	8.320683	14.466146
39	15.870546	14.981609	9.933121	16.081001
40	16.924799	16.368341	10.953651	17.126370
41	16.522377	16.426830	10.581865	16.752430
42	18.419772	17.428608	12.428492	18.604912
43	17.498002	17.299246	11.625706	17.774165
44	20.248551	19.268185	14.460799	20.578019
45	22.803632	21.416920	16.976501	23.112747
46	23.214145	21.439002	17.334003	23.484634
47	25.209951	22.153706	19.450341	25.549267
48	27.950865	24.629205	22.340360	28.399642
49	29.337224	25.694098	23.787176	29.831641
50	31.096879	28.183621	25.480981	31.569679
51	26.780471	22.213687	21.165365	27.201762
52	28.469631	22.739046	22.926543	28.925764
53	29.155229	23.329162	23.669704	29.650242
54	29.932795	24.193668	24.393834	30.399674
55	31.981456	24.760058	26.550597	32.506865

56	33.997725	26.288838	28.609919	34.555425
57	34.841447	27.314339	29.523034	35.457885
58	37.459282	30.356079	32.139280	38.099066
59	37.501643	29.698225	32.311291	38.218271
60	33.692899	28.634824	28.322950	34.314246
61	33.972886	28.633332	28.719361	34.667277
62	34.889739	30.701642	29.225453	35.341304
63	28.735311	26.216323	23.348297	29.372442
64	31.943332	29.595720	26.563955	32.605784
65	34.194553	32.400817	28.614603	34.743376
66	35.269837	33.500474	29.689779	35.822471
67	33.453317	34.163103	27.832719	34.018056
68	34.071425	35.687108	28.200048	34.469763
69	33.871034	36.018348	28.034659	34.306643
70	34.266802	36.783633	28.413496	34.707622
71	32.987125	36.376595	27.117026	33.422220
72	36.155675	38.167232	30.373941	36.643888
73	36.570905	38.410142	30.762082	37.031367
74	36.742391	39.203775	30.843563	37.149031
75	39.133401	40.921750	33.223989	39.524212
76	40.912982	42.139727	34.863107	41.205728
77	38.549846	39.498758	32.690044	38.957914
78	38.520130	39.855362	32.682743	38.940719
79	39.099544	40.812977	33.180247	39.475275
80	36.729444	40.377438	30.942436	37.217702
81	39.192067	43.750569	33.354544	39.666249
82	42.137621	47.538646	36.205809	42.578234
83	44.380716	48.846447	38.657427	44.958732
84	50.090221	53.645848	44.328554	50.641584
85	50.104628	55.414403	44.226166	50.611711
86	48.558913	54.067064	42.810911	49.139964
87	52.020624	59.084010	46.233640	52.618297

88	58.137328	64.997243	52.260281	58.720866
89	56.386440	65.779685	50.986532	57.321296
90	64.761136	78.016179	58.644932	65.323538
91	61.185527	75.908327	55.404463	62.021370
92	61.463634	80.560907	55.814521	62.472467
93	71.421253	91.090324	65.349283	72.217728
94	70.543098	89.181359	64.021080	71.026790
95	60.264605	85.723188	54.455203	61.293788
96	69.607050	94.730087	63.433393	70.397283
97	75.416813	100.072153	69.352307	76.256733
98	76.726920	98.773751	70.148053	77.139138
99	70.636130	98.679003	63.425577	70.548626
100	65.007980	98.089036	57.108823	64.495733
101	68.563566	101.627639	60.912804	68.337482
102	69.502451	101.070800	62.206336	69.514244
103	81.262305	105.076565	74.872626	81.907397
104	86.789741	104.701011	80.382069	87.382838
105	79.874218	100.155711	73.636650	80.542623
106	80.462004	97.333303	74.047992	81.026767
107	67.162695	86.194831	61.431885	68.084002
108	74.750895	91.268623	69.033911	75.669078
109	77.924642	90.587048	71.789686	78.523176
110	67.343169	77.522301	61.113592	67.791484
111	63.454762	74.608431	57.593781	64.121644
112	61.597719	72.644271	55.604690	62.160205
113	63.015032	76.566631	57.246778	63.770892
114	69.934198	82.944202	63.819129	70.455518
115	74.225083	88.912786	68.285093	74.897498
116	82.578758	97.755112	76.261794	83.072940
117	80.095347	96.080857	73.858462	80.642031
118	79.419982	95.539682	73.196985	80.002931
119	81.256782	99.860130	75.028981	81.877180

120	87.984694	105.785453	81.483683	88.467441
121	85.503618	104.569260	78.994466	86.004519
122	92.471194	111.557241	85.752957	92.865330
123	92.576980	110.907591	86.013210	93.058105
124	101.943235	120.955655	94.988947	102.215349
125	108.642637	126.399930	101.509056	108.814382
126	103.817426	122.978463	97.155053	104.308158
127	110.842296	130.929130	104.122044	111.338163
128	117.661817	138.087351	110.556917	117.951693
129	122.832700	143.652784	115.798821	123.172036
130	122.090035	142.560132	115.187126	122.476522
131	133.249716	153.856772	125.378632	133.053487
132	119.572721	136.168988	112.258555	119.612731
133	120.747534	138.756831	112.876360	120.469753
134	110.023686	125.910679	102.369505	109.768416
135	80.039052	90.435896	73.269738	80.122640
136	68.369450	77.524989	61.629056	68.385037
137	70.172648	82.263364	63.520842	70.302361
138	81.174288	95.854955	74.316379	81.253034
139	85.630212	100.557529	78.758874	85.703603
140	86.775518	102.711032	79.970850	86.912189
141	99.007853	111.523516	91.284763	98.541209
142	89.133874	104.430056	82.108252	89.110146
143	93.866361	112.104681	86.766158	93.866527
144	102.406072	121.442225	95.115029	102.324283
145	111.331459	129.383785	103.656433	111.013913
146	106.824685	120.551156	98.921417	106.256362
147	96.703904	112.851162	89.755422	96.748693
148	96.343805	116.955857	89.417634	96.457134
149	101.726827	121.142317	94.507639	101.662424
150	95.514754	117.335593	88.703310	95.729884
151	101.510216	122.669310	94.470291	101.619814

152	110.744630	132.715581	103.878130	111.012465
153	115.712848	138.417836	108.608932	115.847213
154	116.698243	142.624740	109.273998	116.665868
155	119.981390	149.351148	112.725560	120.113828
156	117.151928	148.724638	109.943626	117.364903
157	124.208768	158.565152	116.703337	124.323779
158	126.245655	160.046180	118.679398	126.301611
159	126.687581	161.815336	119.285953	126.857937
160	121.938602	157.223029	114.626381	122.127629
161	129.919747	164.870210	122.083555	129.786727
162	129.676046	162.371585	121.506246	129.311378
163	119.785798	156.085178	112.162368	119.758333
164	106.820678	143.726936	100.096798	107.355791
165	117.948129	155.726483	110.866694	118.341148
166	121.009329	160.927698	113.452617	121.090441
167	118.903209	156.397464	112.167305	119.427419
168	126.679194	167.454775	119.422640	126.907669
169	131.035580	172.126756	123.679753	131.191807
170	137.227849	178.566393	129.786354	137.312184
171	143.880851	184.136896	136.851656	144.160512
172	158.774318	199.459745	150.474312	158.278033
173	141.585536	186.738335	133.693720	141.325756
174	145.627438	193.421628	138.008399	145.543713
175	144.924102	182.153182	137.284409	144.746683
176	138.106971	183.510532	130.958352	138.278175
177	145.934538	194.874461	138.425601	145.967711
178	145.907259	194.963228	138.653071	146.122000
179	139.072685	196.533037	131.890069	139.384627